ASSIGNMENT 9 ERROR CORRECTION	**Name:** Nivetha Dhanakoti **Reg no.:** 3122 22 5001 087

Aim:
To be proficient in developing an application to simulate error correction using hamming code technique using socket programming in C

Write the code using socket programming in C
1. Establish the basic connection between client and the server.
2. Develop an application to simulate error correction using hamming code technique.
3. Client gets input data from user.
4. Find the number of bits to be transmitted.
5. Find the redundant code to be generated depending on the number of bits in the data.
6. Compute the value of r bits.
7. Embed "r" bits in data and send the data to the server.
8. Server will find the r bits and finds whether the data has error or not. If error present, then it will find the location of error.
9. Correct the data if it contains error and display it.

Server Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

void checkAndCorrectHammingCode(int hammingData[], int m, int r);

int main() {
    int serverSock, newSock;
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addr_size;

    // Create socket
    serverSock = socket(AF_INET, SOCK_STREAM, 0);
    if (serverSock < 0) {
        printf("Socket creation failed.\n");
        return -1;
    }

    // Configure server address
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    // Bind socket to the port
    if (bind(serverSock, (struct sockaddr *)&serverAddr, sizeof(serverAddr)) < 0) {
        printf("Bind failed.\n");
        return -1;
    }

    // Start listening for connections
    if (listen(serverSock, 5) < 0) {
        printf("Listen failed.\n");
```

```
        return -1;
    }

    printf("Server listening on port 8080...\n");

    // Accept incoming connection
    addr_size = sizeof(clientAddr);
    newSock = accept(serverSock, (struct sockaddr *)&clientAddr, &addr_size);
    if (newSock < 0) {
        printf("Connection accept failed.\n");
        return -1;
    }

    // Assume maximum size of data + redundant bits
    int m = 4; // Number of data bits (e.g., this can be dynamic depending on client)
    int r = 3; // Number of redundant bits
    int totalBits = m + r;
    int hammingData[totalBits]; // Adjust this size based on the received data

    // Receive Hamming code from client
    recv(newSock, hammingData, totalBits * sizeof(int), 0);
    printf("Received Hamming code from client.\n");

    // Check for errors and correct the data if necessary
    checkAndCorrectHammingCode(hammingData, m, r);

    // Close connection
    close(newSock);
    close(serverSock);
    return 0;
}

void checkAndCorrectHammingCode(int hammingData[], int m, int r) {
    int errorPos = 0, i, j;

    // Calculate the parity bits to detect error
    for (i = 0; i < r; i++) {
        int x = 1 << i;
        int parity = 0;
        for (j = x; j <= m + r; j += 2 * x) {
            for (int k = j; k < j + x && k <= m + r; k++) {
                parity ^= hammingData[k - 1];
            }
        }
        if (parity != 0)
            errorPos += x;
    }

    // If error detected, correct the error
    if (errorPos != 0) {
        printf("Error detected at position: %d\n", errorPos);
        hammingData[errorPos - 1] ^= 1; // Flip the bit to correct error
        printf("Corrected Hamming code: ");
    } else {
        printf("No errors found\n");
```

```
        printf("Hamming code: ");
    }

    // Display the corrected or unchanged Hamming code
    for (i = 0; i < m + r; i++) {
        printf("%d", hammingData[i]);
    }
    printf("\n");
}
```

Client Code:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

void generateHammingCode(int data[], int m, int r, int hammingCode[]);

int main() {
    int sock;
    struct sockaddr_in serverAddr;
    char buffer[1024];

    // Create socket
    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        printf("Socket creation error\n");
        return -1;
    }

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    // Connect to server
    if (connect(sock, (struct sockaddr *)&serverAddr, sizeof(serverAddr)) < 0) {
        printf("Connection failed\n");
        return -1;
    }

    // Get data bits from the user
    int m;
    printf("Enter the number of data bits: ");
    scanf("%d", &m);

    int data[m];
    printf("Enter the data bits (0 or 1):\n");
    for (int i = 0; i < m; i++) {
        printf("Bit %d: ", i + 1);
        scanf("%d", &data[i]);
    }

    // Calculate number of redundant bits
    int r = 0;
    while ((1 << r) < (m + r + 1)) {
        r++;
    }

    int totalBits = m + r;
```

```
    int hammingCode[totalBits];

    // Generate Hamming code
    generateHammingCode(data, m, r, hammingCode);

    printf("Hamming code: ");
    for (int i = 0; i < totalBits; i++) {
        printf("%d", hammingCode[i]);
    }
    printf("\n");

    // Send Hamming code to server
    send(sock, hammingCode, totalBits * sizeof(int), 0);
    printf("Hamming code sent to server.\n");

    // Close the socket
    close(sock);
    return 0;
}

void generateHammingCode(int data[], int m, int r, int hammingCode[]) {
    int totalBits = m + r;
    int dataPos = 0;
    int parityCount = 0;

    // Initialize hammingCode array with data and parity bits
    for (int i = 0; i < totalBits; i++) {
        // Check if current position is power of 2 (parity bit position)
        if ((i + 1) == (1 << parityCount)) {
            hammingCode[i] = 0;  // Initialize parity bits to 0
            parityCount++;
        } else {
            hammingCode[i] = data[dataPos];  // Place data bits
            dataPos++;
        }
    }

    // Calculate parity bits
    for (int i = 0; i < r; i++) {
        int parityPos = (1 << i);  // Position of current parity bit
        int parityValue = 0;

        // Calculate parity for current position
        for (int j = parityPos; j <= totalBits; j += 2 * parityPos) {
            for (int k = j; k < j + parityPos && k <= totalBits; k++) {
                parityValue ^= hammingCode[k - 1];  // XOR the data bits covered by this parity bit
            }
        }

        hammingCode[parityPos - 1] = parityValue;  // Set the parity bit
    }
}
```

Output 1 :

```
nivethadhanakoti@Nivethas-MacBook-Pro A9 % ./server
Server listening on port 8080...
Received Hamming code from client.
No errors found
Hamming code: 0110011
```

```
nivethadhanakoti@Nivethas-MacBook-Pro A9 % ./client
Enter the number of data bits: 4
Enter the data bits (0 or 1):
Bit 1: 1
Bit 2: 0
Bit 3: 1
Bit 4: 1
Hamming code: 0110011
Hamming code sent to server.
```

Output 2 :

```
nivethadhanakoti@Nivethas-MacBook-Pro A9 % ./server
 Server listening on port 8080...
 Received Hamming code from client.
 Error detected at position: 2
 Corrected Hamming code: 0110011
```

```
nivethadhanakoti@Nivethas-MacBook-Pro A9 % ./client
Enter the number of data bits: 7
Enter the data bits (0 or 1):
Bit 1: 1
Bit 2: 0
Bit 3: 1
Bit 4: 1
Bit 5: 1
Bit 6: 0
Bit 7: 1
Hamming code: 00100110101
Hamming code sent to server.
```

Learning Outcomes:

I have learnt to be proficient in developing an application to simulate error correction using hamming code technique using socket programming in C